Trabalho Final — Projeto de Arquitetura de Software

Sistema de Gestão de Planos de Voo

Ana Clara Purper Bianca Piassini Maria Melloni Tarciso Mota

7 de outubro de 2026

1 Introdução

O sistema proposto tem como objetivo prover o backend e a arquitetura para a criação, validação, controle de ciclo de vida e reserva de recursos para planos de voo. O domínio é composto por 19 elementos — 15 classes, 1 enumeração e 3 objetos de valor — organizados em torno do agregado central **PlanoDeVoo**. O sistema oferece 11 casos de uso ao ator **Operador de Voo**.

As entidades de infraestrutura (aeroportos, aerovias, aeronaves e pilotos) são tratadas como dados pré-existentes para consulta e planejamento. Este documento está estruturado em duas partes principais:

1. **Diagrama de Domínio e Casos de Uso:** Modelagem conceitual do domínio em UML, dicionário de classes, decisões arquiteturais de modelagem, casos de uso detalhados e matriz de rastreabilidade.
2. **Diagramas C4 Model e Estrutura do Projeto:** Visão arquitetural em múltiplos níveis de abstração (Contexto, Containers e Componentes) e a organização estrutural de pacotes seguindo os princípios de Arquitetura Limpa (Clean Architecture).

Convenções adotadas:

- Horários usam `LocalDateTime` e distâncias usam quilômetros. Velocidades estão em km/h, altitudes em pés e permanências em minutos.
- Atributos marcados como **derivados** (`/nome`) são calculados e não informados pelo usuário.
- Cada diagrama possui o código PlantUML correspondente para geração visual em ferramentas como `https://www.planttext.com`.

2 Parte 1 — Diagrama de Domínio e Casos de Uso

2.1 Diagrama de domínio

O **PlanoDeVoo** concentra o ciclo de vida e é dono do percurso (Trechos e Pousos) e das Reservas. As regras de categoria ficam fora da hierarquia de aeronaves, numa estratégia (`RegraOperacao`).

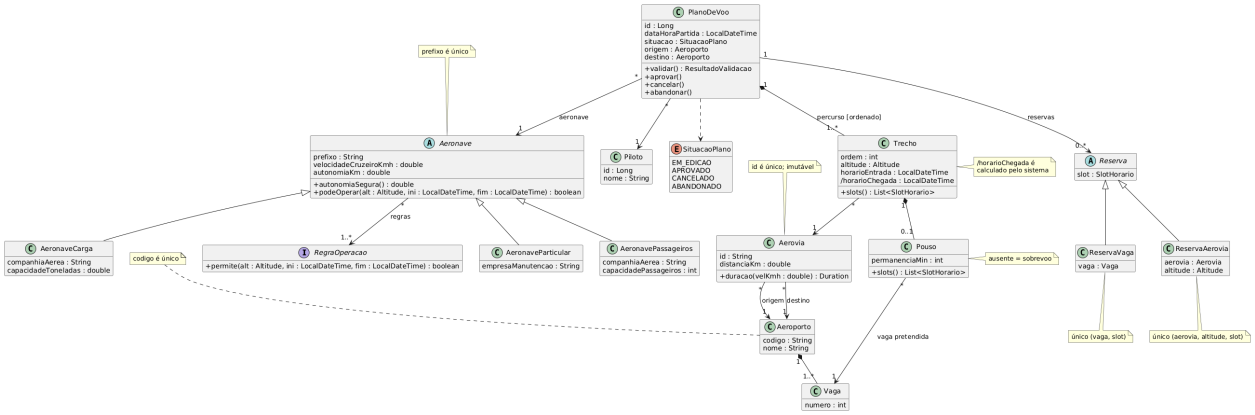


Figura 1: Modelo de domínio: 15 classes, 1 enumeração e 3 objetos de valor (tipos de atributo).

Leia de cima para baixo: o plano compõe trechos; cada trecho usa uma aerovia e pode terminar em pouso numa vaga; a aprovação gera reservas por slot de 1 hora. `Altitude`, `SlotHorario` e `ResultadoValidacao` são objetos de valor e aparecem como tipos de atributo.

Código PlantUML equivalente:

```
@startuml
skinparam classAttributeIconSize 0
hide empty members

interface RegraOperacao {
  +permite(alt : Altitude, ini : LocalDateTime, fim : LocalDateTime) : boolean
}

abstract class Aeronave {
  prefixo : String
  velocidadeCruzeiroKmh : double
  autonomiaKm : double
  +autonomiaSegura() : double
  +podeOperar(alt : Altitude, ini : LocalDateTime, fim : LocalDateTime) : boolean
}
note top of Aeronave : prefixo é único

class AeronaveParticular {
  empresaManutencao : String
}

class AeronavePassageiros {
  companhiaAerea : String
  capacidadePassageiros : int
}

class AeronaveCarga {
  companhiaAerea : String
  capacidadeToneladas : double
}

Aeronave <|-- AeronaveParticular
Aeronave <|-- AeronavePassageiros
Aeronave <|-- AeronaveCarga
Aeronave "*" --> "1..*" RegraOperacao : regras

class Piloto {
  id : Long
  nome : String
}

enum SituacaoPlano {
```

```
  EM_EDICAO
  APROVADO
  CANCELADO
  ABANDONADO
}

class PlanoDeVoo {
  id : Long
  dataHoraPartida : LocalDateTime
  situacao : SituacaoPlano
  origem : Aeroporto
  destino : Aeroporto
  +validar() : ResultadoValidacao
  +aprovar()
  +cancelar()
  +abandonar()
}

PlanoDeVoo "*" --> "1" Piloto
PlanoDeVoo "*" --> "1" Aeronave : aeronave
PlanoDeVoo ..> SituacaoPlano

class Trecho {
  ordem : int
  altitude : Altitude
  horarioEntrada : LocalDateTime
  /horarioChegada : LocalDateTime
  +slots() : List<SlotHorario>
}
note right of Trecho : /horarioChegada é\ncalculado pelo sistema

class Pouso {
  permanenciaMin : int
  +slots() : List<SlotHorario>
}

PlanoDeVoo "1" *-- "1..*" Trecho : percurso [ordenado]
Trecho "1" *-- "0..1" Pouso
note right of Pouso : ausente = sobrevoo

class Aerovia {
  id : String
  distanciaKm : double
  +duracao(velKmh : double) : Duration
}
note top of Aerovia : id é único; imutável

class Aeroporto {
  codigo : String
  nome : String
}
note top of Aeroporto : codigo é único

class Vaga {
  numero : int
}

Trecho "*" --> "1" Aerovia
Aerovia "*" --> "1" Aeroporto : origem
Aerovia "*" --> "1" Aeroporto : destino
Aeroporto "1" *-- "1..*" Vaga
Pouso "*" --> "1" Vaga : vaga pretendida

abstract class Reserva {
  slot : SlotHorario
```

```
}

class ReservaAerovia {
  aerovia : Aerovia
  altitude : Altitude
}

class ReservaVaga {
  vaga : Vaga
}

Reserva <|-- ReservaAerovia
Reserva <|-- ReservaVaga
PlanoDeVoo "1" --> "0..*" Reserva : reservas
note bottom of ReservaAerovia : único (aerovia, altitude, slot)
note bottom of ReservaVaga : único (vaga, slot)
@enduml
```

2.2 Dicionário das classes

Cada classe concentra uma responsabilidade do enunciado. As regras ficam junto dos dados que elas usam.

Classe	Tipo	Atributos principais	Responsabilidade
Aeroporto	Entidade	codigo (único), nome	Ponto da malha aérea; agrega suas vagas
Vaga	Entidade	numero	Posição de estacionamento de um aeroporto; recurso reservável
Aerovia	Entidade imutável	id (único), distanciaKm	Ligação orientada origem → destino; calcula a duração do voo para uma velocidade
Altitude	Objeto de valor	pes (25.000–35.000, passo 1.000)	Garante que só existam níveis de voo válidos
Piloto	Entidade	id, nome	Responsável pelo plano (sem regras próprias)
Aeronave	Entidade abstrata	prefixo (único), velocidadeCruzeiroKmh, autonomiaKm	Calcula a autonomia segura (85%) e delega a regra de operação
AeronaveParticular	Subclasse	empresaManutencao	Categoria particular
AeronavePassageiros	Subclasse	companhiaAerea, capacidadePassageiros	Categoria comercial de passageiros
AeronaveCarga	Subclasse	companhiaAerea, capacidadeToneladas	Categoria comercial de carga
RegraOperacao	Interface (Strategy)	—	Diz se uma aeronave pode usar uma altitude num intervalo; uma implementação por regra
PlanoDeVoo	Entidade (raiz de agregado)	id, dataHoraPartida, situacao	Controla o ciclo de vida, guarda o percurso e as reservas e valida o plano
SituacaoPlano	Enumeração	EM_EDICAO, APROVADO, CANCELADO, ABANDONADO	Estados possíveis do plano
Trecho	Entidade (parte do plano)	ordem, altitude, horarioEntrada, /horarioChegada	Um uso de aerovia no percurso; calcula a chegada e os slots ocupados
Pouso	Objeto de valor	permanenciaMin	Parada num aeroporto, com a vaga pretendida; ausência = sobrevoo

Classe	Tipo	Atributos principais	Responsabilidade
SlotHorario	Objeto de valor	inicio (hora cheia)	Janela de 1 hora usada no controle de ocupação
Reserva	Entidade abstrata	slot	Ocupação efetivada na aprovação
ReservaAerovia	Subclasse	aerovia, altitude	Ocupa uma altitude de uma aerovia num slot
ReservaVaga	Subclasse	vaga	Ocupa uma vaga num slot
ResultadoValidacao	Objeto de valor	valido, erros[]	Lista todas as violações encontradas, não só a primeira

2.3 Decisões de modelagem

Sete decisões moldam o modelo. Para cada uma registramos a escolha, a alternativa considerada e o motivo.

#	Decisão adotada	Alternativa considerada	Justificativa
D1	Aeronave abstrata com subclasses por categoria, e a regra de altitude/horário extraída para a interface **RegraOperacao** (Strategy)	Uma classe Aeronave com campo `categoria` e `if/switch` nas regras	O enunciado avisa que virão novas categorias e regras. Com Strategy, uma regra nova é uma classe nova, sem mexer nas existentes (aberto/fechado)
D2	**PlanoDeVoo** é a raiz do agregado: Trechos e Pousos só existem dentro dele e só mudam através dele	Trechos como entidades independentes, editadas direto pela API	Garante a regra "somente planos em edição podem ser modificados" num só lugar
D3	**Pouso** opcional no Trecho: sem pouso = sobrevoo	Enum `tipoChegada` + campos de vaga/permanência que podem ficar nulos	Evita estados inválidos (sobrevoo com vaga). A regra de pouso obrigatório no destino fica na validação
D4	**horarioChegada** é derivado (distância ÷ velocidade); o usuário informa só a entrada	Usuário informa entrada e chegada e o sistema compara	Uma única fonte de verdade. A validação compara a entrada do trecho seguinte com a chegada calculada
D5	**Uma Reserva por slot de 1 hora**, com unicidade (aerovia, altitude, slot) e (vaga, slot)	Uma reserva por intervalo (início–fim) e checagem de sobreposição	A unicidade pode virar restrição no banco. Dois planos concorrentes não conseguem reservar o mesmo slot, e a aprovação fica atômica
D6	**Aerovia imutável** e referenciada por id	Permitir editar distância ou excluir	Exigência do enunciado: preserva os planos e reservas que já usam a aerovia
D7	**Vaga** é uma entidade composta pelo Aeroporto; a quantidade de vagas é implícita pela composição	Um atributo `quantidadeVagas` no Aeroporto e referência por número	O plano indica *a vaga pretendida* e a reserva aponta para uma vaga específica. Modelar como entidade permite unicidade (vaga, slot) diretamente no banco

2.4 Diagrama de casos de uso

O sistema concentra-se no ciclo de vida e planejamento de voos. O ator **Operador de Voo** interage com a API REST através de 11 casos de uso, onde a aprovação (UC07) inclui obrigatoriamente a validação (UC06).

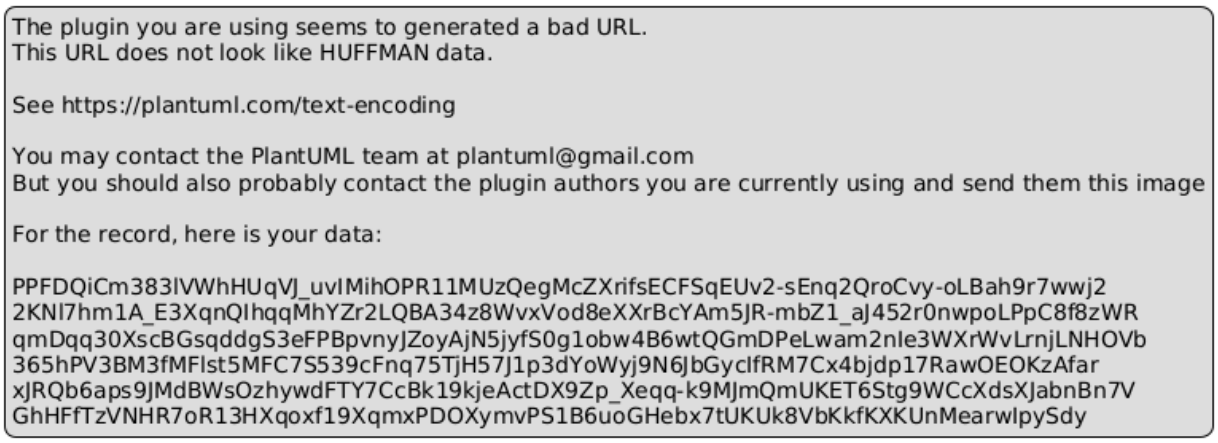


Figura 2: Casos de uso: 1 ator (Operador de Voo), 11 casos e 1 «include».

O «include» mostra que a aprovação repete toda a validação com a disponibilidade do momento exato, conforme a seção 6.5 do enunciado.

Código PlantUML equivalente:

```
@startuml
left to right direction
actor "Operador de Voo" as Operador
rectangle "Sistema de Gestão de Planos de Voo" {
  usecase "UC01 Consultar malha e recursos" as UC01
  usecase "UC02 Criar plano" as UC02
  usecase "UC03 Editar percurso" as UC03
  usecase "UC04 Ver altitudes livres" as UC04
  usecase "UC05 Ver vagas livres" as UC05
  usecase "UC06 Validar plano" as UC06
  usecase "UC07 Aprovar plano" as UC07
  usecase "UC08 Consultar plano" as UC08
  usecase "UC09 Listar por situação" as UC09
  usecase "UC10 Abandonar plano" as UC10
  usecase "UC11 Cancelar plano" as UC11
}
Operador --> UC01
Operador --> UC02
Operador --> UC03
Operador --> UC04
Operador --> UC05
Operador --> UC06
Operador --> UC07
Operador --> UC08
Operador --> UC09
Operador --> UC10
Operador --> UC11
UC07 .> UC06 : <<include>>
@enduml
```

2.5 Casos de uso identificados

Foram identificados 11 casos de uso para o **Operador de Voo**. As entidades de infraestrutura (aeroportos, aerovias, aeronaves e pilotos) são consultadas como recursos pré-existentes. Cada caso de uso corresponde a um ou mais endpoints da API REST.

ID	Caso de uso	Ator	Regra-chave
UC01	Consultar malha e recursos	Operador	Consulta de aeroportos, aerovias, pilotos e frota
UC02	Criar plano de voo	Operador	Nasce em edição, sem trechos
UC03	Editar percurso do plano	Operador	Só em edição; não reserva recursos
UC04	Consultar altitudes disponíveis em aerovia	Operador	Situação do momento; não reserva
UC05	Consultar vagas disponíveis em aeroporto	Operador	Situação do momento; não reserva
UC06	Validar plano	Operador	Não muda estado nem reserva
UC07	Aprovar plano	Operador	«include» UC06; reserva tudo ou nada
UC08	Consultar plano por identificador	Operador	Qualquer situação
UC09	Listar planos por situação	Operador	Inclui cancelados e abandonados
UC10	Abandonar plano	Operador	Só em edição
UC11	Cancelar plano	Operador	Só aprovado com partida futura; libera reservas

UC03 agrupa as operações de edição: adicionar, alterar e remover trechos, definir altitude, sobrevoo ou pouso, vaga e permanência. Se necessário, pode ser granularizado em UC03a (incluir trecho), UC03b (alterar trecho) e UC03c (remover trecho).

2.6 Descrição dos casos de uso principais

Os seis casos de uso abaixo concentram as regras de negócio do ciclo de vida do plano de voo.

UC02 — Criar plano de voo

- **Ator:** Operador de Voo
- **Pré-condição:** piloto, aeronave e aeroportos de origem e destino cadastrados.
- **Fluxo principal:** operador informa dados básicos; sistema valida referências e instancia o plano com situação `EM_EDICAO`, sem trechos.
- **Exceções:** referência inexistente (404); origem igual ao destino (422).
- **Pós-condição:** plano em edição, sem reservas.

UC03 — Editar percurso do plano

- **Ator:** Operador de Voo
- **Pré-condição:** plano `EM_EDICAO`.
- **Fluxo principal:** inclusão, alteração ou remoção de trechos; cálculo dinâmico de duração e horário de chegada.
- **Exceções:** plano fora de edição (409); aerovia ou vaga inexistente (404); altitude inválida (422).
- **Pós-condição:** percurso atualizado sem realizar reservas.

UC06 — Validar plano

- **Ator:** Operador de Voo
- **Pré-condição:** plano `EM_EDICAO`.
- **Fluxo principal:** verificação completa de percurso contínuo, horários e duração, compatibilidade de categoria da aeronave (regras de operação), limite de 85% de autonomia segura e disponibilidade de slots de aerovias e vagas.
- **Exceções:** plano fora de edição (409).
- **Pós-condição:** emissão do laudo de validação contendo a totalidade das violações (sem alterar estado ou criar reservas).

UC07 — Aprovar plano

- **Ator:** Operador de Voo
- **Pré-condição:** plano `EM_EDICAO`.
- **Fluxo principal:** executa UC06 atomicamente em transação; caso válido, persiste as instâncias de `ReservaAerovia` e `ReservaVaga`, transicionando o plano para `APROVADO`.
- **Exceções:** violação de regra (422); conflito concorrente de slot (409 com rollback total).
- **Pós-condição:** plano aprovado com garantia de unicidade de reservas.

UC10 — Abandonar plano

- **Ator:** Operador de Voo
- **Pré-condição:** plano `EM_EDICAO`.
- **Fluxo principal:** transiciona o estado para `ABANDONADO`.
- **Pós-condição:** plano congelado para histórico; sem reservas a liberar.

UC11 — Cancelar plano

- **Ator:** Operador de Voo
- **Pré-condição:** plano `APROVADO` com data/hora de partida futura.
- **Fluxo principal:** libera em transação única todas as reservas associadas e transiciona para `CANCELADO`.
- **Exceções:** voo já ocorrido ou plano não aprovado (409).
- **Pós-condição:** plano cancelado e slots liberados.

2.7 Rastreabilidade: regras do enunciado → modelo

Regra (seção do enunciado)	Onde fica no modelo	Caso de uso
Aerovias orientadas e imutáveis (2)	Aerovia: origem e destino distintos, sem operações de alteração	UC01, UC03
Particular só a 25.000–27.000 pés (3)	RegraOperacao da categoria particular	UC06, UC07
Passageiros só a partir de 28.000 pés (3)	RegraOperacao da categoria passageiros	UC06, UC07
Carga só entre 00:00 e 06:00, todo o trecho (3)	RegraOperacao da categoria carga	UC06, UC07
Autonomia com margem de 15%, reinicia no pouso (3, 6.3)	Aeronave.autonomiaSegura() + PlanoDeVoo.validar()	UC06
Percurso contínuo da origem ao destino (4.1)	PlanoDeVoo.validar()	UC06
Chegada = entrada + distância ÷ velocidade (4.1)	Trecho./horarioChegada (derivado)	UC03, UC06
Pouso obrigatório no destino final (4.2)	Último Trecho deve ter Pouso	UC06
Altitudes de 25.000 a 35.000, passo 1.000 (5.1)	Objeto de valor Altitude	UC03, UC04
Slots de 1 h alinhados à hora cheia (5.1, 5.2)	SlotHorario; Trecho.slots() e Pouso.slots()	UC04, UC05, UC06
Altitude/vaga não pode ser usada por dois planos no mesmo slot (5.1, 5.2)	Unicidade em ReservaAerovia e ReservaVaga	UC07
Aprovação tudo ou nada (5.4, 6.5)	Transação em PlanoDeVoo.aprovar()	UC07
Cancelamento libera reservas, só se a partida for futura (6.8)	PlanoDeVoo.cancelar()	UC11

Regra (seção do enunciado)	Onde fica no modelo	Caso de uso
Voo começa e termina no mesmo dia; permanência final pode passar da meia-noite (5.5)	PlanoDeVoo.validar()	UC06
Só planos em edição podem ser modificados (6.8)	PlanoDeVoo como raiz do agregado verifica a situação	UC03, UC10

3 Parte 2 — Diagramas C4 Model e Estrutura do Projeto

3.1 Visão Geral da Arquitetura

Para representar a arquitetura do sistema em diferentes níveis de granularidade, foi adotado o modelo **C4 Model** (Contexto, Containers, Componentes e Código). A arquitetura foi concebida seguindo os princípios de **Arquitetura Limpa (Clean Architecture / Ports and Adapters)**, garantindo o isolamento das regras de negócio do domínio e a independência de frameworks externos, bibliotecas de persistência ou interfaces com o usuário.

3.2 C4 Nível 1: Diagrama de Contexto

O Diagrama de Contexto estabelece os limites do sistema, apresentando seus usuários e como ele se posiciona no ambiente operacional. O ator principal é o **Operador de Voo**, que interage diretamente com o sistema para planejar, validar e aprovar planos de voo.

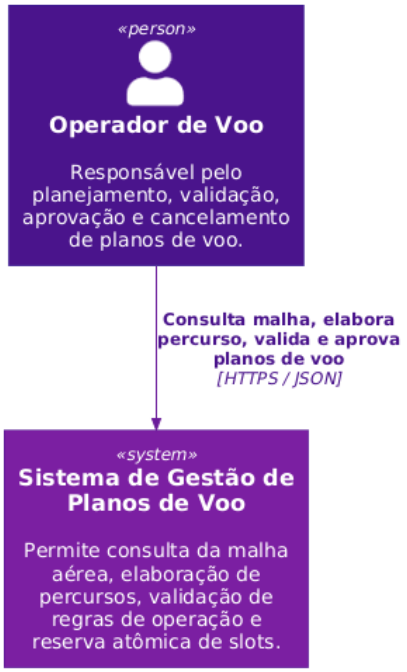


Figura 3: C4 Nível 1: Diagrama de Contexto do Sistema de Gestão de Planos de Voo.

Código PlantUML (C4-PlantUML):

```
@startuml
!include https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Context.puml

UpdateElementStyle("person", $bgColor="#4A148C", $fontColor="#ffffff", $borderColor="#311B92")
UpdateElementStyle("system", $bgColor="#7B1FA2", $fontColor="#ffffff", $borderColor="#4A148C")
UpdateRelStyle($lineColor="#6A1B9A", $textColor="#4A148C")
```

```
Person(operador, "Operador de Voo", "Responsável pelo planejamento, validação, aprovação e
    cancelamento de planos de voo.")
System(sistemaVoos, "Sistema de Gestão de Planos de Voo", "Permite consulta da malha aérea, elabora
    ção de percursos, validação de regras de operação e reserva atômica de slots.")

Rel(operador, sistemaVoos, "Consulta malha, elabora percurso, valida e aprova planos de voo", "
    HTTPS / JSON")
@enduml
```

3.3 C4 Nível 2: Diagrama de Containers

O Diagrama de Containers detalha as unidades de execução de software que compõem o sistema:

- **Single Page Application (SPA):** Interface web moderna executada no navegador do Operador de Voo.
- **Backend API (Spring Boot / Java):** Aplicação responsável pela execução dos casos de uso, validações de domínio e controle transacional.
- **Banco de Dados Relacional (PostgreSQL):** Responsável pelo armazenamento persistente com constraints de integridade e unicidade para as reservas de slots.



Figura 4: C4 Nível 2: Diagrama de Containers da Solução.

Código PlantUML (C4-PlantUML):

```
@startuml
!include https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Container.puml

UpdateElementStyle("person", $bgColor="#4A148C", $fontColor="#ffffff", $borderColor="#311B92")
UpdateElementStyle("container", $bgColor="#7B1FA2", $fontColor="#ffffff", $borderColor="#4A148C")
UpdateElementStyle("containerDb", $bgColor="#311B92", $fontColor="#ffffff", $borderColor="#1A237E")
UpdateRelStyle($lineColor="#6A1B9A", $textColor="#4A148C")

Person(operador, "Operador de Voo", "Usuário do sistema")

System_Boundary(c1, "Sistema de Gestão de Planos de Voo") {
    Container(spa, "Single Page Application", "React / TypeScript", "Interface web interativa para
    operadores desenharem e validarem percursos de voo.")
```

```
    Container(api, "Backend API", "Java / Spring Boot", "Disponibiliza endpoints REST para os 11
    casos de uso, motor de validação e controle transacional de reservas.")
    ContainerDb(db, "Banco de Dados Relacional", "PostgreSQL", "Armazena malha, frotas, planos,
    trechos e registros de reservas de slots com restrições de unicidade.")
}

Rel(operador, spa, "Opera planos de voo e monitora disponibilidade", "HTTPS")
Rel(spa, api, "Chamadas de API", "JSON / HTTPS")
Rel(api, db, "Lê e grava dados e reservas", "JDBC / JPA")
@enduml
```

3.4 C4 Nível 3: Diagrama de Componentes (Backend API)

O Diagrama de Componentes descreve a organização interna do container **Backend API**. A divisão segue as camadas da Clean Architecture:

- **Controllers (Interface Adapters - Inbound):** Tratam requisições HTTP, convertem payloads e invocam os Casos de Uso.
- **Use Cases (Application Layer):** Coordenam os fluxos dos 11 casos de uso identificados, garantindo a transacionalidade nas operações de aprovação e cancelamento.
- **Domain Model (Core):** Entidades e serviços de domínio que contêm as regras de negócio puras (ex: cálculo de slots, validações de percurso e estratégias de categoria).
- **Repositories / Gateways (Interface Adapters - Outbound):** Portas e adaptadores para persistência em banco de dados relacional.

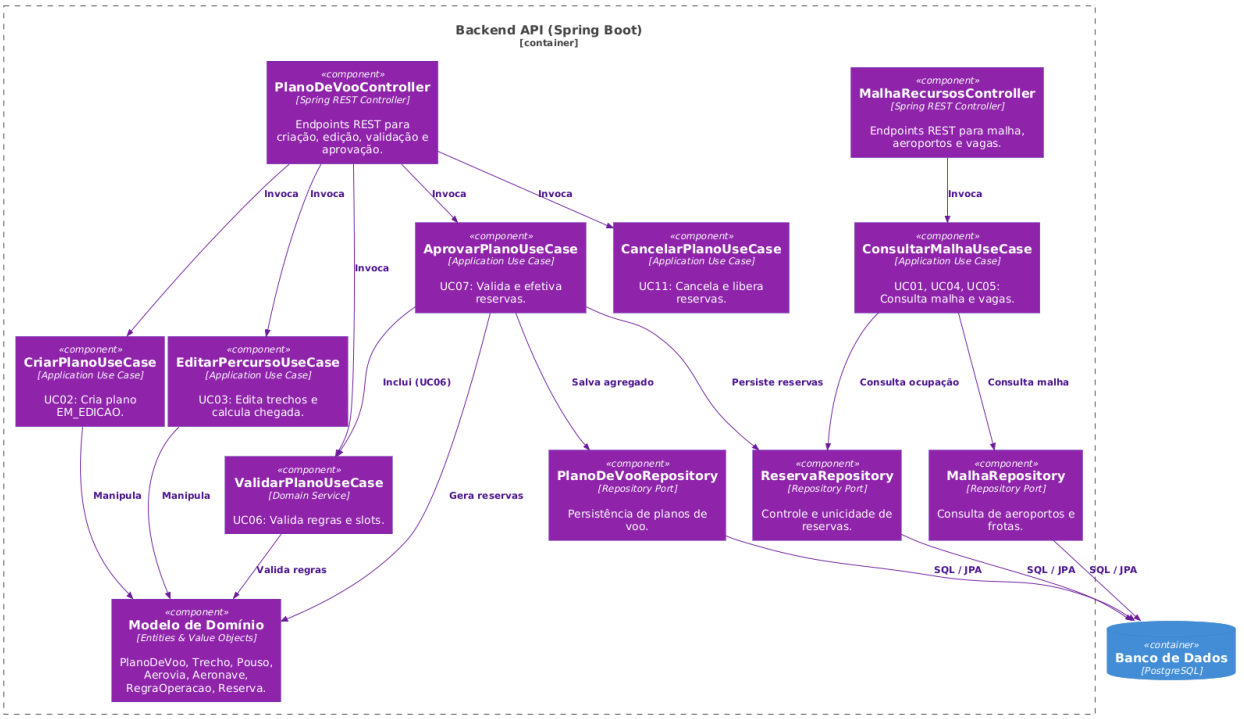


Figura 5: C4 Nível 3: Diagrama de Componentes do Backend API.

Código PlantUML (C4-PlantUML):

```
@startuml
!include https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Component.puml

UpdateElementStyle("component", $bgColor="#8E24AA", $fontColor="#ffffff", $borderColor="#5E35B1")
UpdateElementStyle("containerDb", $bgColor="#311B92", $fontColor="#ffffff", $borderColor="#1A237E")
UpdateRelStyle($lineColor="#6A1B9A", $textColor="#4A148C")
```

```
Container_Boundary(api, "Backend API (Spring Boot)") {
    Component(planoCtrl, "PlanoDeVooController", "Spring REST Controller", "Endpoints REST para
    criação, edição, validação e aprovação.")
    Component(malhaCtrl, "MalhaRecursosController", "Spring REST Controller", "Endpoints REST para
    malha, aeroportos e vagas.")

    Component(criarPlanoUC, "CriarPlanoUseCase", "Application Use Case", "UC02: Cria plano
    EM_EDICAO.")
    Component(editarPercursoUC, "EditarPercursoUseCase", "Application Use Case", "UC03: Edita
    trechos e calcula chegada.")
    Component(validarPlanoUC, "ValidarPlanoUseCase", "Domain Service", "UC06: Valida regras e slots
    .")
    Component(aprovarPlanoUC, "AprovarPlanoUseCase", "Application Use Case", "UC07: Valida e
    efetiva reservas.")
    Component(cancelarPlanoUC, "CancelarPlanoUseCase", "Application Use Case", "UC11: Cancela e
    libera reservas.")
    Component(consultarMalhaUC, "ConsultarMalhaUseCase", "Application Use Case", "UC01, UC04, UC05:
     Consulta malha e vagas.")

    Component(dominio, "Modelo de Domínio", "Entities & Value Objects", "PlanoDeVoo, Trecho, Pouso,
     Aerovia, Aeronave, RegraOperacao, Reserva.")

    Component(planoRepo, "PlanoDeVooRepository", "Repository Port", "Persistência de planos de voo
    .")
    Component(reservaRepo, "ReservaRepository", "Repository Port", "Controle e unicidade de
    reservas.")
    Component(malhaRepo, "MalhaRepository", "Repository Port", "Consulta de aeroportos e frotas.")
}

ContainerDb(db, "Banco de Dados", "PostgreSQL")

Rel(planoCtrl, criarPlanoUC, "Invoca")
Rel(planoCtrl, editarPercursoUC, "Invoca")
Rel(planoCtrl, validarPlanoUC, "Invoca")
Rel(planoCtrl, aprovarPlanoUC, "Invoca")
Rel(planoCtrl, cancelarPlanoUC, "Invoca")
Rel(malhaCtrl, consultarMalhaUC, "Invoca")

Rel(aprovarPlanoUC, validarPlanoUC, "Inclui (UC06)")
Rel(criarPlanoUC, dominio, "Manipula")
Rel(editarPercursoUC, dominio, "Manipula")
Rel(validarPlanoUC, dominio, "Valida regras")
Rel(aprovarPlanoUC, dominio, "Gera reservas")

Rel(aprovarPlanoUC, planoRepo, "Salva agregado")
Rel(aprovarPlanoUC, reservaRepo, "Persiste reservas")
Rel(consultarMalhaUC, reservaRepo, "Consulta ocupação")
Rel(consultarMalhaUC, malhaRepo, "Consulta malha")

Rel(planoRepo, db, "SQL / JPA")
Rel(reservaRepo, db, "SQL / JPA")
Rel(malhaRepo, db, "SQL / JPA")
@enduml
```

3.5 Estrutura de Pastas do Projeto

A estrutura de diretórios foi desenhada para refletir estritamente as fronteiras arquiteturais da **Clean Architecture**, mantendo o núcleo de domínio completamente desacoplado de dependências tecnológicas e frameworks.

```
src/
|-- main/
|   |-- java/
|   |   |-- br/pucrs/projarc/gestaoplanosvoo/
```

```
|   |       |-- domain/                 # Camada de Dominio (Pura, sem frameworks)
|   |       |   |-- model/              # Entidades e Objetos de Valor
|   |       |   |   |-- PlanoDeVoo.java
|   |       |   |   |-- Trecho.java
|   |       |   |   |-- Pouso.java
|   |       |   |   |-- Aerovia.java
|   |       |   |   |-- Aeroporto.java
|   |       |   |   |-- Vaga.java
|   |       |   |   |-- Piloto.java
|   |       |   |   |-- SituacaoPlano.java
|   |       |   |   |-- Altitude.java
|   |       |   |   |-- SlotHorario.java
|   |       |   |   |-- ResultadoValidacao.java
|   |       |   |   |-- aeronave/       # Hierarquia e Strategy de Aeronaves
|   |       |   |   |   |-- Aeronave.java
|   |       |   |   |   |-- AeronaveParticular.java
|   |       |   |   |   |-- AeronavePassageiros.java
|   |       |   |   |   |-- AeronaveCarga.java
|   |       |   |   |   \-- RegraOperacao.java
|   |       |   |   \-- reserva/        # Agregado e Subclasses de Reserva
|   |       |   |       |-- Reserva.java
|   |       |   |       |-- ReservaAerovia.java
|   |       |   |       \-- ReservaVaga.java
|   |       |   \-- repository/         # Portas de Saida (Interfaces de Repositorio)
|   |       |       |-- PlanoDeVooRepositoryPort.java
|   |       |       |-- ReservaRepositoryPort.java
|   |       |       \-- MalhaRepositoryPort.java
|   |       |
|   |       |-- application/            # Camada de Aplicacao (Casos de Uso)
|   |       |   |-- dto/                # DTOs de Requisicao e Resposta
|   |       |   |   |-- request/
|   |       |   |   \-- response/
|   |       |   \-- usecases/           # Implementacao dos 11 Casos de Uso
|   |       |       |-- CriarPlanoUseCase.java
|   |       |       |-- EditarPercursoUseCase.java
|   |       |       |-- ValidarPlanoUseCase.java
|   |       |       |-- AprovarPlanoUseCase.java
|   |       |       |-- CancelarPlanoUseCase.java
|   |       |       |-- AbandonarPlanoUseCase.java
|   |       |       |-- ConsultarPlanoUseCase.java
|   |       |       |-- ListarPlanosPorSituacaoUseCase.java
|   |       |       \-- ConsultarDisponibilidadeUseCase.java
|   |       |
|   |       \-- infrastructure/         # Camada de Infraestrutura e Adaptadores
|   |           |-- api/                # Adaptadores de Entrada (REST Controllers)
|   |           |   |-- PlanoDeVooController.java
|   |           |   |-- MalhaRecursosController.java
|   |           |   \-- exception/      # Exception Handlers HTTP (404, 409, 422)
|   |           |-- persistence/        # Adaptadores de Saida (JPA / PostgreSQL)
|   |           |   |-- entities/       # Entidades JPA de mapeamento ORM
|   |           |   |-- repositories/   # Interfaces Spring Data JPA
|   |           |   \-- adapters/       # Implementacoes das Portas de Repositorio
|   |           \-- config/             # Configuracoes de Beans e Injecao
|   \-- resources/
|       |-- application.yml             # Configuracoes da aplicacao Spring Boot
|       \-- db/migration/               # Migracoes versionadas de banco (Flyway)
|           \-- V1__criar_tabelas_iniciais.sql
```

Detalhamento das Camadas do Projeto

- **`domain/`:** O coração do sistema. Contém todas as regras de negócio puras (ex: validação de continuidade do percurso, cálculo de autonomia com margem de 15%, aplicação das estratégias de altitude por categoria de aeronave). Não possui nenhuma anotação de frameworks (como

Spring ou JPA).

- **`application/`:** Orquestra os fluxos de execução através dos Casos de Uso. Não acessa o banco de dados diretamente, mas sim através das interfaces de portas (*Ports*) definidas no domínio.
- **`infrastructure/`:** Contém os adaptadores externos:
  - **`api`:** Controladores REST do Spring MVC que recebem JSON, validam requisições e formatam respostas HTTP de acordo com o status apropriado (200, 201, 404, 409, 422).
  - **`persistence`:** Mapeamentos ORM (JPA/Hibernate) e repositórios Spring Data que persistem os dados garantindo a atomicidade transacional e constraints de unicidade para as reservas.